Week 3 Lab: Proportional Motor-Speed Control

ENGR 3303 – Controls Engineering
Fall 2026

Main goal. Implement a closed-loop proportional controller on the encoder motor, measure its performance, investigate how the proportional gain K_p changes the response, and select a reasonable K_p for your motor.

What Changes from Week 2?

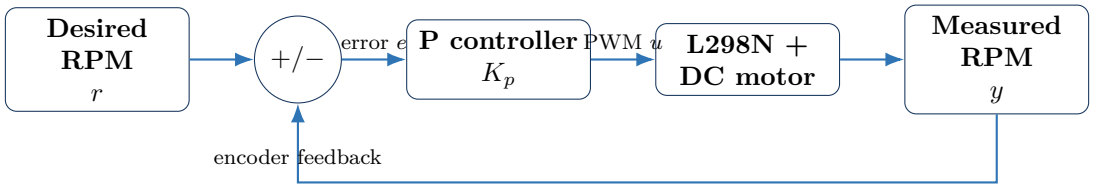
In Week 2, PWM was chosen directly and the encoder was used only to measure the result.

In Week 3, the encoder measurement becomes part of the controller:

$$\boxed{e(t) = r(t) - y(t)}$$

$$\boxed{u(t) = K_p e(t)}$$

The Arduino repeats this calculation during operation.



Hardware and Software

Use the same system from Weeks 1 and 2:

- Arduino Uno or compatible board;
- L298N motor-driver module;
- 12 V encoder gearmotor;
- external motor power supply;
- laptop with Arduino IDE;
- Python.

Safety

- Turn motor power OFF before changing wiring.
- Keep the wheel clear of fingers, wires, clothing, and tools.
- Do not intentionally stall the motor.
- Keep Arduino GND and L298N GND connected.
- Stop increasing K_p if the motor becomes rough, strongly oscillatory, or repeatedly changes direction unexpectedly.

Python Package Setup

Install the free Python packages before lab:

```
pip install pyserial pandas numpy matplotlib scipy
```

Pin Assignment

Use the same wiring as the previous labs.

Function	Arduino pin	Connection
Motor PWM	D9	L298N ENA
Motor direction 1	D7	L298N IN1
Motor direction 2	D8	L298N IN2
Encoder A	D2	yellow encoder lead
Encoder B	D3	green encoder lead
Encoder VCC	5 V	blue encoder lead
Common ground	GND	black encoder lead and L298N GND

Course encoder calibration: Use `COUNTS_PER_OUTPUT_REV = 494.0` for Encoder A counted with `RISING` edges only. Recalibrate only if the motor or encoder-counting method changes.

Lab 3A – Implement Proportional Control

Suggested time: 75 minutes

Part 1 – Choose a Reference Speed

Your instructor will assign or approve a reference speed.

A reasonable first test is approximately

$$r = 120 \text{ RPM}.$$

Record your value:

$$\boxed{r = _____ \text{ RPM}}$$

Do not choose a reference above the reliable speed capability of your motor.

Part 2 – Understand the Controller

At each controller update:

$$e = r - y.$$

Then

$$u = K_p e.$$

Because Arduino PWM is limited to 0–255,

$$\boxed{u_{\text{actual}} = \text{constrain}(K_p e, 0, 255).}$$

For this lab, the motor is commanded only in the forward direction. If the measured speed becomes greater than the reference and $e < 0$, the PWM command is clipped to zero rather than reversing the motor.

Part 3 – Predict the Initial PWM

At startup,

$$y(0) \approx 0.$$

Therefore,

$$e(0) \approx r.$$

For your first proportional gain K_p,

$$u(0) = K_p r.$$

Complete:

$$K_p = _____$$

$$u(0) = (_____)(_____) = _____$$

Will the initial command saturate at PWM 255?

Yes / No

Part 4 – Upload the P-Controller Code

Open:

`week3_p_controller.ino`

Before uploading, verify:

- `REFERENCE_RPM`;
- `KP`;
- `COUNTS_PER_OUTPUT_REV`.

The controller update period is

$$T_s = 0.10 \text{ s}.$$

Every 0.10 s, the Arduino:

1. calculates RPM;
2. calculates error;
3. calculates the proportional command;
4. limits PWM to 0–255;
5. updates the motor;
6. sends data to Python.

The output data contain:

time, reference, RPM, error, PWM

Part 5 – Record the First Closed-Loop Test

Close Arduino Serial Monitor before starting Python.
Open:

`week3_record_p_control.py`

Change the COM port to your Arduino port.
Run the Python script.
Python will:

1. connect to Arduino;

2. wait for the experiment to begin;
3. record the controller data;
4. save the data as a CSV file;
5. create response plots.

   Use a descriptive filename such as

   `P_Kp1.csv`.

> **Lab 3A checkoff:** Show the instructor a plot containing the desired RPM and measured RPM, plus a second plot showing the PWM command.

Part 6 – Interpret the First Test

Record:

Quantity	Result
Reference RPM	
K_p	
Steady-state RPM	
Steady-state error	
Maximum PWM	
Did PWM reach 255?	
Response smooth?	

Lab 3B – Tune the Proportional Gain

Suggested time: 75 minutes

Part 1 – Test Three Gains

Keep the same reference speed.

Test three instructor-approved values of K_p.

Suggested first-pass values are:

$$\boxed{K_p = 1, \quad 3, \quad 6}$$

but adjust these values if your Week 2 motor data show that another range is more appropriate.

For every gain:

1. change `KP` in Arduino;
2. upload the program;
3. run the Python data recorder;
4. save the data using a different filename;
5. inspect the response before increasing K_p.

> If the motor becomes strongly oscillatory, repeatedly slams between PWM limits, or behaves unexpectedly, stop the test and notify the instructor. Do not automatically continue to a larger gain.

Part 2 – Compare the Three Tests

Complete the table using your measured data.

K_p	Final RPM	Error (RPM)	Approx. settling time	Max PWM	Observation

Calculate steady-state error from

$$\boxed{e_{ss} = r - y_{ss}.}$$

Part 3 – Python Comparison Plot

Use

`week3_compare_kp.py`

to plot all three measured RPM responses on the same axes.
Your plot should include:

- desired RPM;
- response for low K_p;
- response for medium K_p;
- response for high K_p.

Create a second plot comparing the PWM commands.

Part 4 – Compare with the Week 2 Model

Use your Week 2 motor model

$$G(s) = \frac{K}{\tau s + 1}.$$

For each proportional gain,

$$\boxed{T(s) = \frac{K_p G(s)}{1 + K_p G(s)}.}$$

In Python, compute the ideal closed-loop response directly:

```
import numpy as np
import matplotlib.pyplot as plt

Kmotor = ...    # Week 2 result
tau = ...       # Week 2 result
Kp = ...
R = ...         # reference RPM

dc_gain = (Kmotor*Kp)/(1 + Kmotor*Kp)
tau_cl = tau/(1 + Kmotor*Kp)

t = np.linspace(0, 5, 400)
rpm_model = R*dc_gain*(1 - np.exp(-t/tau_cl))

plt.plot(t, rpm_model)
plt.grid(True)
plt.xlabel("Time (s)")
plt.ylabel("Motor Speed (RPM)")
plt.show()
```

Compare the ideal prediction with the measured motor response.

Part 5 – Theoretical Steady-State Error

Using your Week 2 motor gain K,

$$\boxed{e_{ss,\text{ideal}} = \frac{r}{1 + KK_p}.}$$

For each K_p, calculate:

K_p	Ideal error	Measured error	Difference

Do not expect the values to match perfectly because the real motor includes saturation, friction, encoder noise, and other effects not included in the simple model.

Part 6 – Select a Reasonable Gain

Choose one K_p from your tests as the best proportional gain for your motor.

$$\boxed{K_{p,\text{selected}} = _____}$$

Explain your choice using data.
Consider:

- steady-state error;
- response speed;
- PWM saturation;
- smoothness;
- repeatability.

Part 7 – Optional Disturbance Demonstration

With instructor approval only, observe what happens when a small external load is applied safely to the motor system.

Record qualitatively:

1. Does RPM decrease?
2. What happens to the error?
3. What happens to PWM?
4. Does the motor recover completely to the reference?

Do not touch the spinning wheel directly with your hand.

Python Data Analysis

For one run, the CSV file contains:

Time (s)	Reference (RPM)	RPM	Error (RPM)	PWM
0.10	0	0	0	0
⋮	⋮	⋮	⋮	⋮
2.10	120	35	85	255
2.20	120	62	58	174
⋮	⋮	⋮	⋮	⋮

The numbers above are examples only.
The actual CSV data are generated automatically by Python from the Arduino serial stream.

Questions to Submit

Answer briefly.

1. What reference speed did your team use?
2. What three K_p values did you test?
3. For each K_p, what steady-state RPM and steady-state error did you measure?
4. How did increasing K_p affect steady-state error?
5. How did increasing K_p affect response speed?
6. Which tests caused PWM saturation?
7. Why does pure proportional control generally leave a nonzero steady-state error?
8. Which K_p did your team select and why?
9. How did the measured responses differ from the Week 2 model predictions?
10. Give two practical reasons why a very large K_p may be undesirable.
11. Why will adding integral control in Week 4 help with the remaining steady-state error?

What to Submit

One submission per team:

- CSV data for all three K_p tests;
- measured RPM comparison plot;
- PWM comparison plot;

- completed gain-comparison table;
- ideal vs. measured steady-state-error calculations;
- selected K_p with justification;
- answers to Questions 1–11;
- Arduino code;
- Python recording and analysis code.

End-of-Lab Shutdown

Before leaving:

1. set PWM to zero;
2. turn OFF motor power;
3. save all CSV, Arduino, and Python files;
4. disconnect the USB cable after motor power is off;
5. return the hardware in an organized condition.